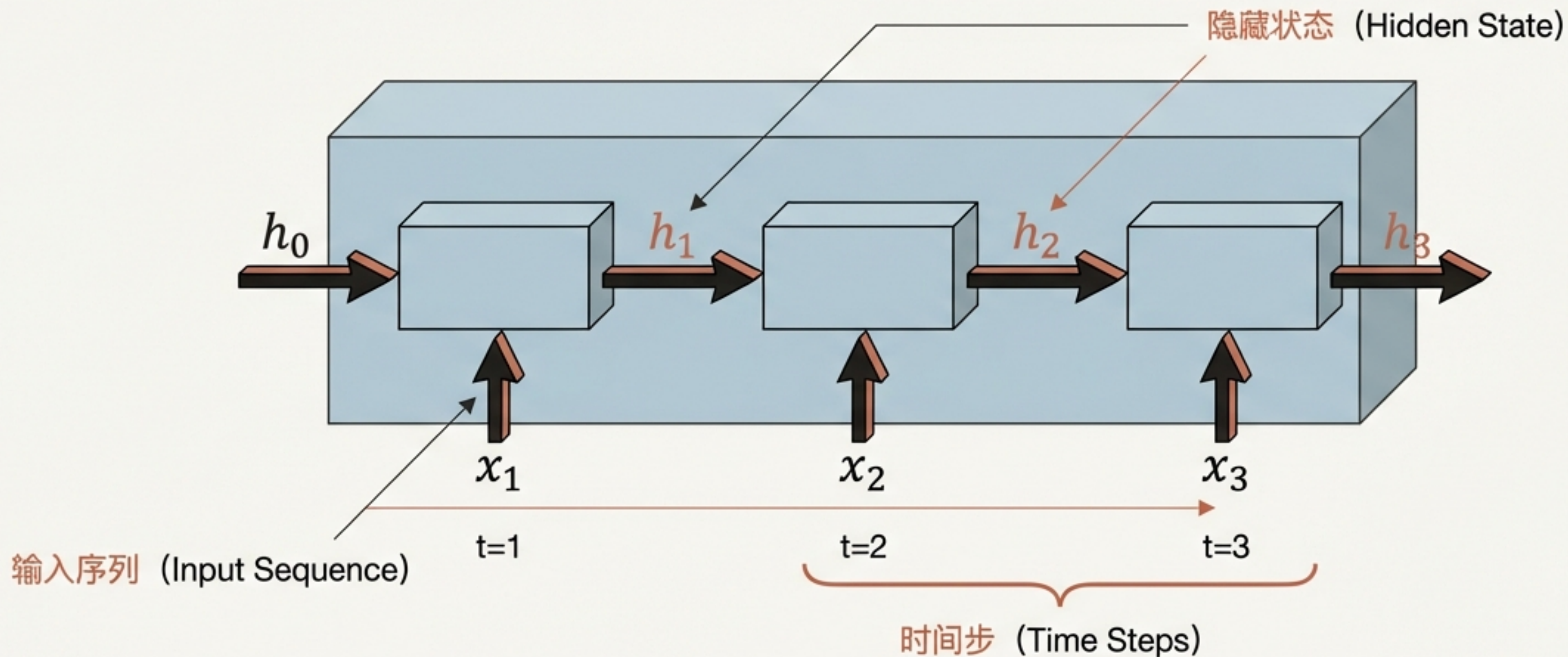


``num_layers=2``... 这究竟意味着什么？

```
multi_layer_rnn = nn.RNN(input_size=100,  
                           hidden_size=200,  
                           num_layers=2,  
                           dropout=0.1)
```

在PyTorch的`nn.RNN`中，`num\_layers`参数有什么作用？
当我们设置多于一个层时，网络内部到底发生了什么？

回顾基础：一个RNN层如何处理序列



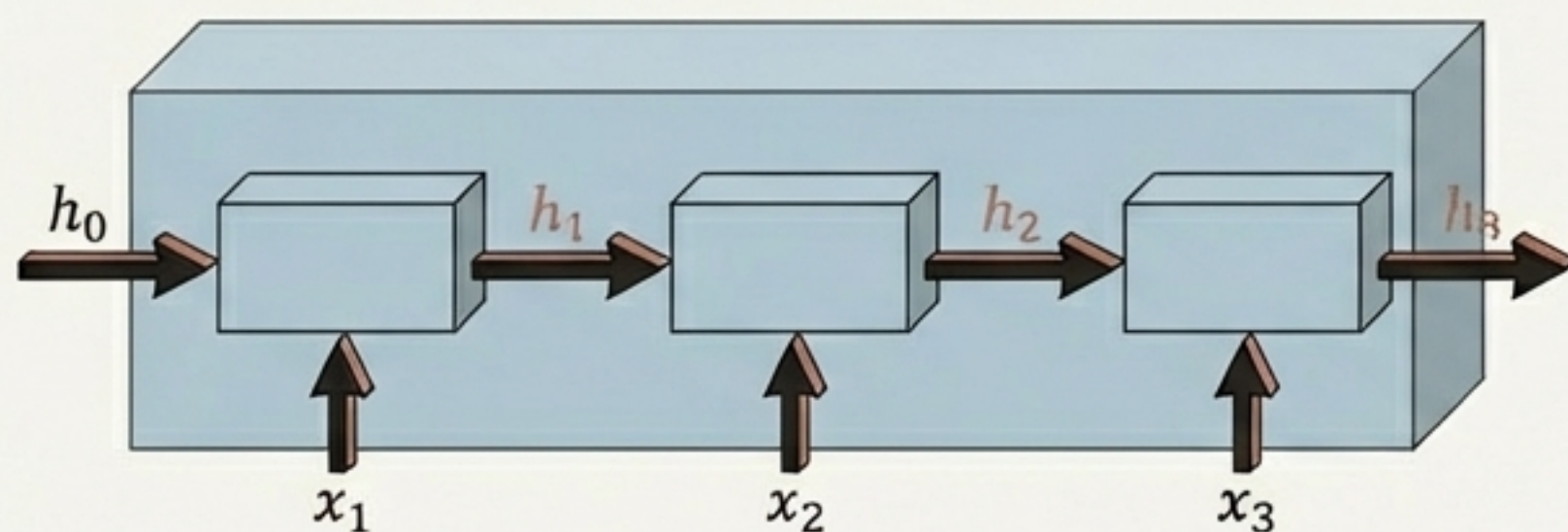
核心思想：信息在时间维度上“水平”传递，捕捉序列中的时序依赖关系。

在一个时间步 (t) 内, RNN的核心计算

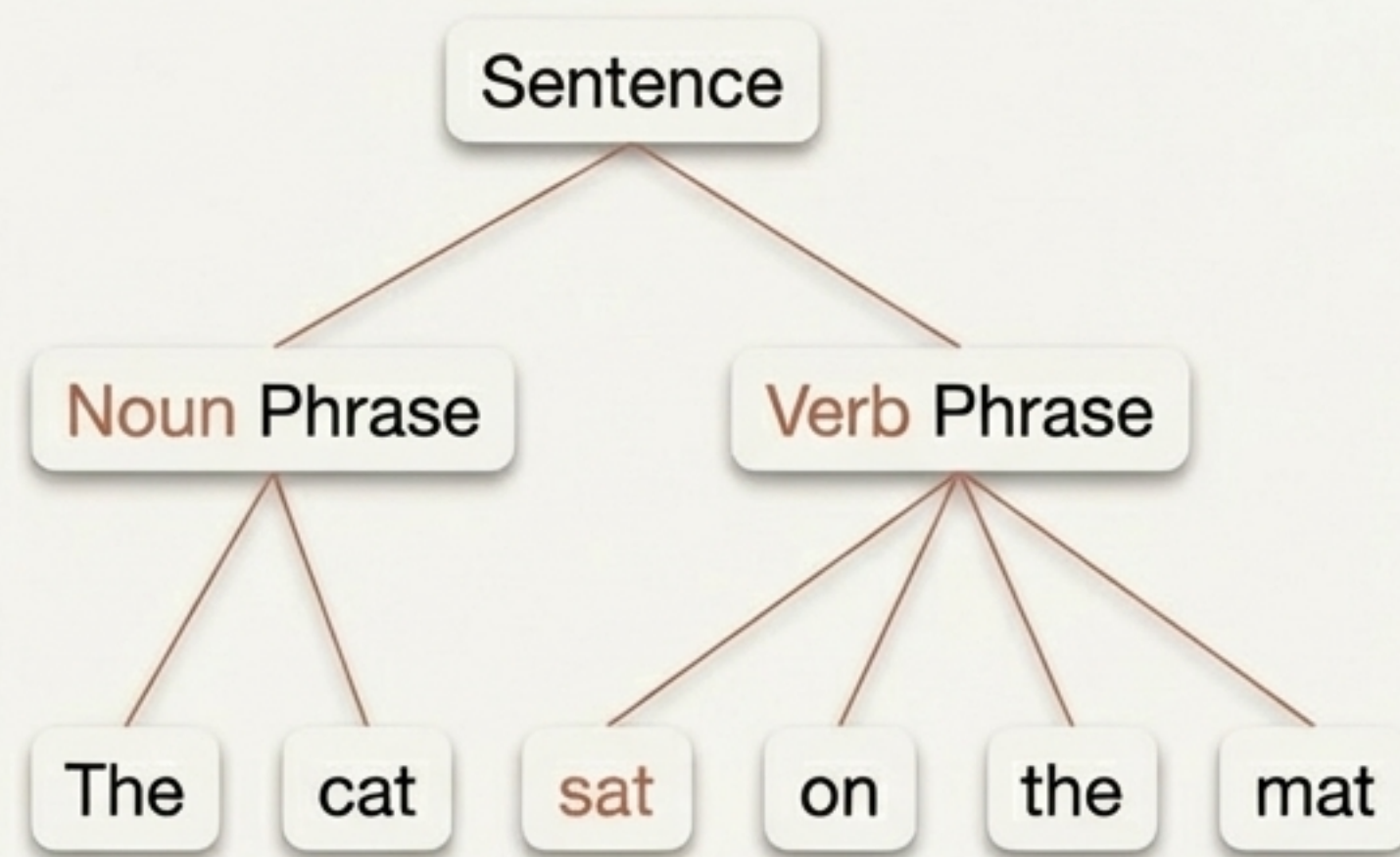


在每个时间步, **RNN单元**结合当前输入和**过去的记忆**, 生成一个新的、包含上下文信息的隐藏状态。

单一抽象层级的局限性：为什么需要更“深”？ 如果数据中的模式本身就具有层次结构呢？



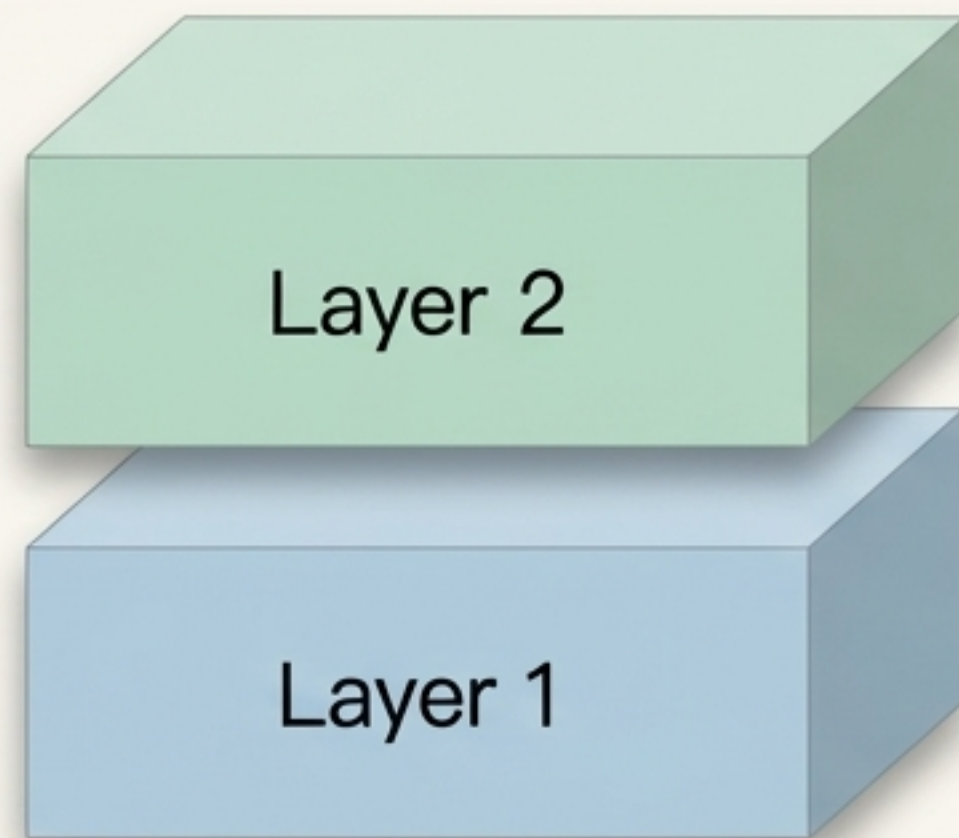
学习直接的时序模式
(Learning direct temporal patterns)



一个单层RNN擅长在单一抽象层面上捕捉序列依赖。但对于更复杂的任务（如语言理解、视频分析），我们需要模型能够学习到‘**模式中的模式**’——即层次化的特征表示。

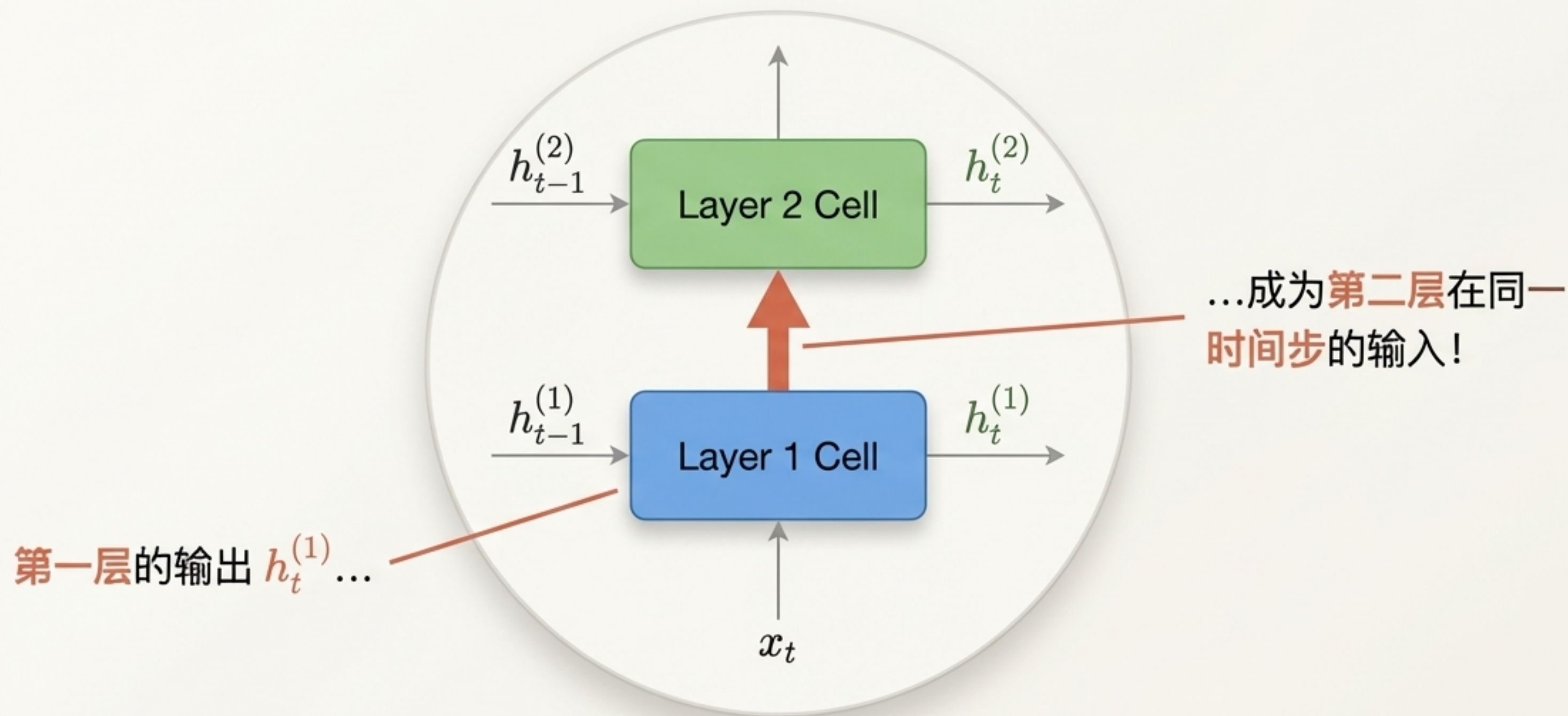
解决方案：“堆叠”RNN层以构建深度

```
multi_layer_rnn = nn.RNN(input_size=100,  
                           hidden_size=200,  
                           num_layers=2,  
                           dropout=0.1)
```



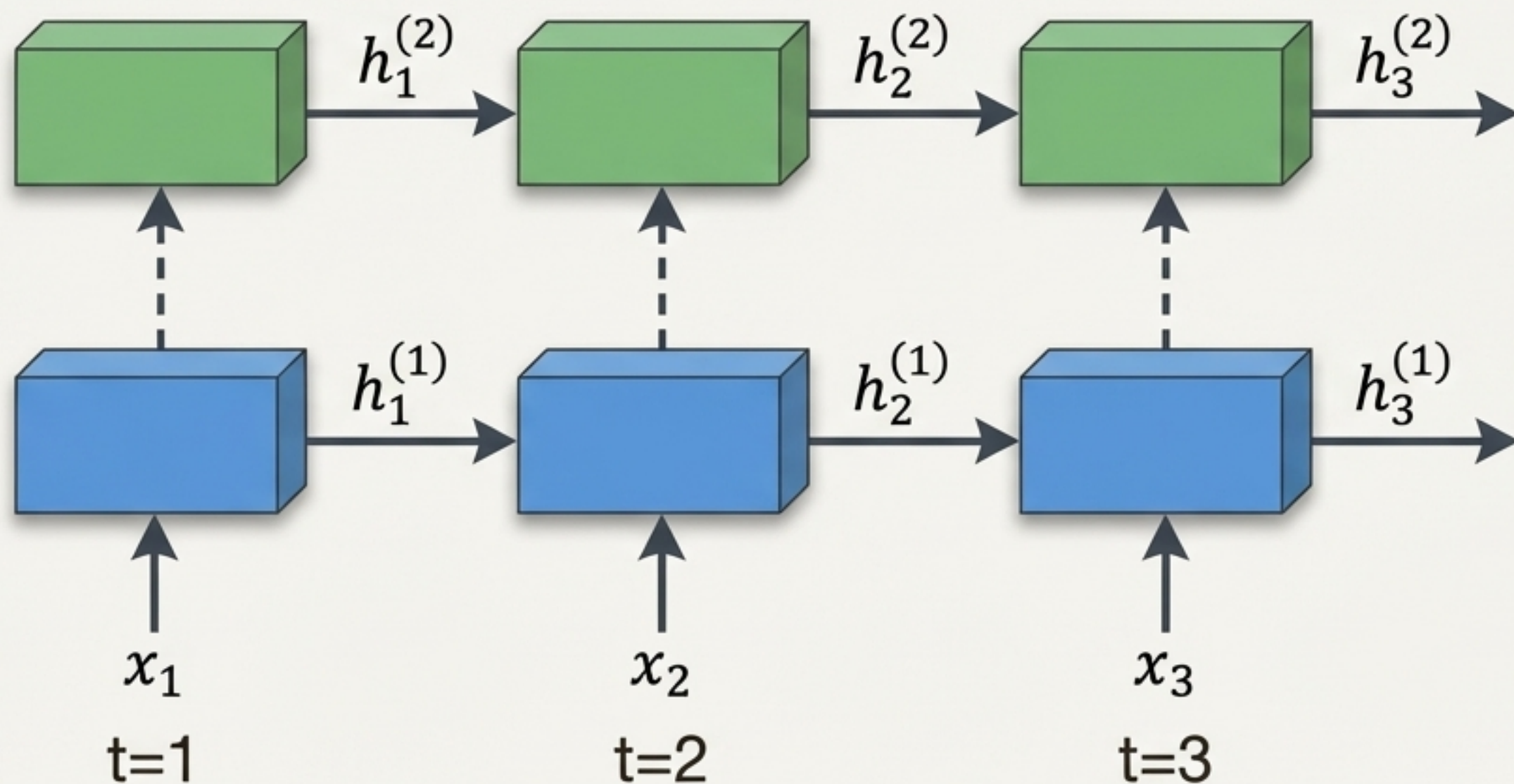
`num_layers=2` 的含义就是将两个独立的RNN层堆叠在一起。这为网络增加了“深度”，使其能够处理不同层次的抽象信息。

关键环节：信息如何在层与层之间“垂直”流动



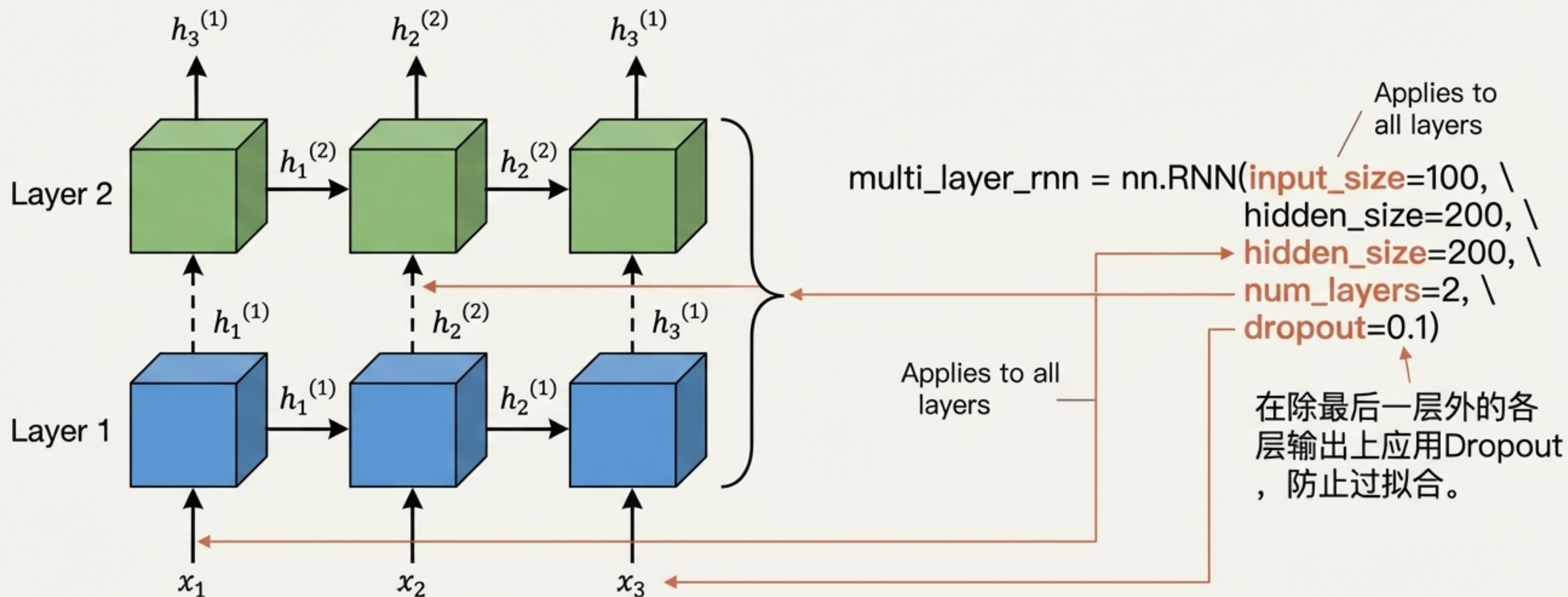
在任意时间步 t ，第 L 层的隐藏状态 $h_t^{(L)}$ 会作为第 $L + 1$ 层的输入。
这就是信息在网络深度上“垂直”传递的方式。

全局视角：一个双层RNN的完整数据流



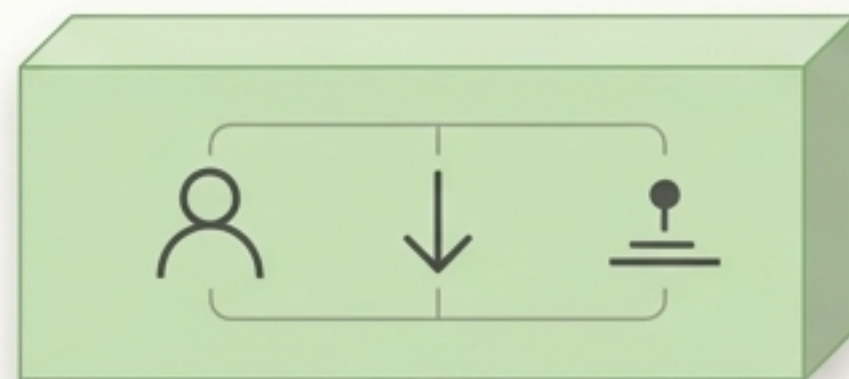
网络同时在两个维度上处理信息：沿**时间轴**（水平方向）捕捉**时序动态**，沿网络**深度**（垂直方向）提取更高层次的特征。

从概念到代码：参数的最终解读



“深度”的力量：堆叠RNN为何如此有效？

第二层：在第一层特征的基础上，
构建短语和句子级别的语义表示



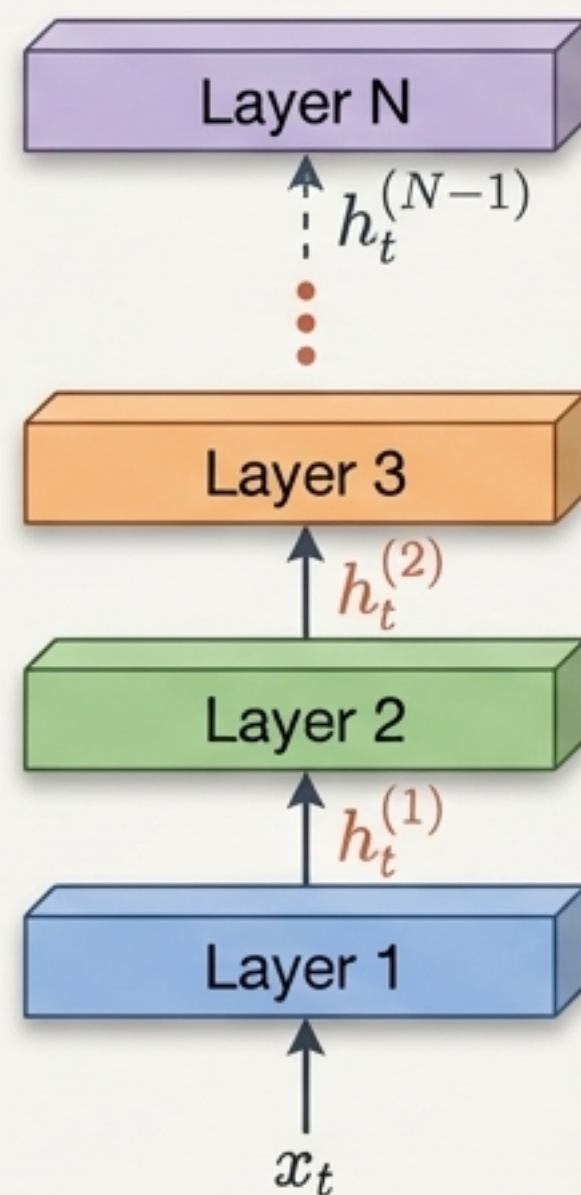
第一层：学习词汇和基础语法模式



The cat sat on the mat

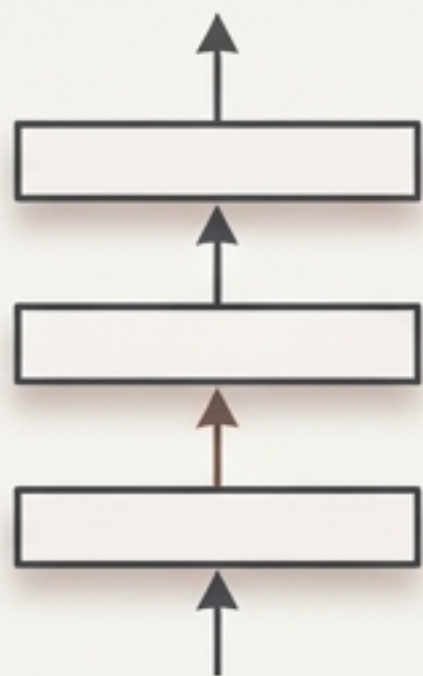
通过堆叠，网络能够构建一个**特征层次结构**（**Hierarchy of Features**）。底层网络处理原始数据，高层网络则处理底层网络提炼出的、更抽象的特征。

超越两层： $\text{num_layers} > 2$ 的通用模式

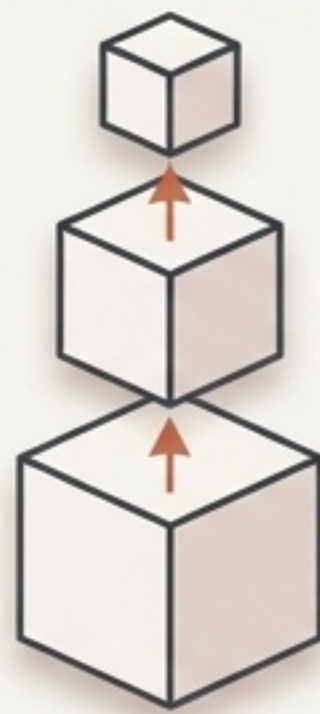


这个模式可以推广到任意数量的层。每一层都以前一层在同一时间步的隐藏状态作为输入，从而逐步提升特征的抽象级别。然而，层数过多也可能导致梯度消失和过拟合问题。

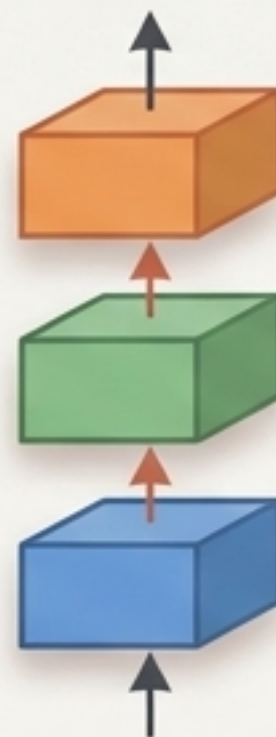
“深度”是跨架构的通用语言



Stacked MLP



Deep CNN



Stacked RNN

无论是用于图像的**CNN**，还是用于序列的**RNN**，增加“深度”（堆叠层）都是提升模型表达能力、学习从具体到抽象的特征表示的核心策略。理解 **num_layers**，就是理解了深度学习中一个最基本且强大的思想。